SEMESTER THEME

Project Proposals

Integrating AI into Applications

Nine product ideas, each built around an LLM-powered core, with use cases to implement and room for your own creativity.



Systems Analysis and Design

Shiraz University • Fall 2026

Welcome

Nine product ideas. One theme. Your choice.

THIS SEMESTER'S THEME

Integrating AI into Applications

Large language models (LLMs) can read, write, summarize and reason about text. Through a simple API call, any application can borrow those abilities.

This semester you will analyze, design and build a product-sized application in which the LLM is the **core**, not a decoration. Each of the nine projects below is a real product idea with **required use cases**, an **AI core** to design, and plenty of room for your **own creativity**.

HOW TO READ A PROJECT (TWO PAGES EACH)

WHAT IS IT?
The idea in simple words, plus the full feature set of a product-sized app.

USERS AND FLOW
Who uses it, a three-step journey, and what makes it feel like a real product.

SUGGESTED TECH STACK
A recommended choice for each part of the system, with a short reason.

USE CASES
The list your team must analyze, design and implement. A sparkle marks LLM use cases.

AI CORE
The LLM-powered heart of the app, an example flow, and what to watch out for.

MAKE IT YOURS
Creative directions. Originality is a major part of your grade.

HOW TO CHOOSE

PICK WHAT YOU LIKE
Choose an app you would enjoy using and showing to others.

PICK WHAT YOU LEARN
Look at the stack: choose technologies you want to practice.

PICK THE AI IDEA
Which AI core excites you most? Start from there.

Good to know. Tech stacks are suggestions: any language or framework is fine. The listed use cases are the minimum, so extend them freely. The dots on each banner show how much work to expect: more dots, more work.

Overview

All nine projects at a glance.

01 Smart Trip Planner
Plan a whole trip from a single sentence.
Itinerary generation & chat

02 Fridge to Recipe
Turn a fridge photo into tonight's recipes.
Fridge photo to recipes

03 Study Battle
Real-time quiz battles on your own notes.
Questions & answer judging

04 Explain It Differently
Any concept, explained the way you understand.
Personalized explanations

05 Gamified Learning Application
Study with XP, streaks, quests and an AI tutor.
Quizzes & AI tutor

06 Local Multiplayer Party Games
Pass-the-phone party games with an AI host.
Fresh packs & game master

07 Mood-Based Music Discovery
Describe a mood, get songs that fit it.
Mood to music matching

08 Language Learning Application
Flashcards, exercises and AI conversation.
Tutor, coach & conversation

09 Digital Closet & Outfit Planner
Catalog your clothes; an AI stylist plans outfits.
Photo tagging & stylist

Smart Trip Planner

Tell it where. It plans the rest.

01

DIFFICULTY ●●●●●

AI CORE Itinerary generation & chat

WHAT IS IT?

A trip-planning app where you describe your dream trip in plain words (“4 days in Istanbul, relaxed, food and history”) and get a full day-by-day itinerary you can edit.

It checks timing and budget, shows everything on a map, and re-plans when something changes.

MAIN FEATURES

- **Trip workspace:** destinations, dates, budget, interests
- **AI itinerary** with times, travel gaps and cost estimates
- **Drag-and-drop editing** with automatic schedule checks
- **Map view** of each day’s stops with route estimates
- **Budget tracker:** estimated vs. real spending per category
- **Packing list** built from the weather and planned activities
- **Shared trips:** plan together with friends
- **Offline view and PDF export** of the final plan

HOW IT WORKS

1. **Describe**
   Tell the app about your trip in plain words.
2. **Generate**
   Get a full draft itinerary on a map.
3. **Refine**
   Edit, chat with the planner, share it.

WHO USES IT

- **Traveler**
  Plans trips, edits the itinerary, tracks the budget.
- **Travel buddy**
  Joins a shared trip to view, comment or edit.
- **Admin**
  Manages place data; watches AI usage and abuse.

SUGGESTED TECH STACK

Frontend	**Next.js + React, Leaflet or Mapbox**	Interactive maps and drag-and-drop day planning.
Backend	**NestJS** or **FastAPI**	Clean modules for trips, itineraries, sharing and AI calls.
Database	**PostgreSQL (+ PostGIS)**	Trips, stops and locations, plus geographic queries.
Services	**Places API, Open-Meteo, Redis cache**	Real places and weather; cache to save cost and time.
Testing & DevOps	**Playwright, Docker, CI**	End-to-end tests of generate, edit and share flows.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

✓ Accounts & login ✓ Trips saved in a database ✓ Maps and real place data ✓ Input validation
✓ Loading & error states ✓ Offline view ✓ Rate limits on AI calls

Use Cases & AI Core

Smart Trip Planner

01

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Create a trip** Set destination, dates, travelers, budget and interests.	Traveler
UC2	**Generate an itinerary from free text** ✨ Describe the trip in words and get a structured day-by-day plan.	Traveler
UC3	**Edit the itinerary** Add, move, remove and reorder stops; timing is re-validated.	Traveler
UC4	**Chat with the planner** ✨ "Make day 2 less tiring" actually changes the plan, not just the chat.	Traveler
UC5	**Track the budget** Log expenses and compare them with the estimates per category.	Traveler
UC6	**Generate a packing list** ✨ Based on weather, trip length and planned activities.	Traveler
UC7	**Share and collaborate** Invite a friend to view or edit; changes appear for both.	Traveler, buddy
UC8	**Review AI usage** See requests, cost and failed generations; block abusive accounts.	Admin

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Itinerary generator:** free text becomes a structured plan (days, stops, times) as validated JSON.
- **Grounded in real data:** every suggested place is matched against a places API, nothing is invented.
- **Conversational editing:** the chat uses tool calling to move, replace or remove stops.
- **Smart re-planning:** rain or a closed museum? The LLM proposes alternatives within budget.
- **Daily briefing:** a short summary, local tips and packing advice for each day.

EXAMPLE FLOW

INPUT	"4 days in Istanbul, relaxed, food + history, medium budget"
PROMPT	"You are a trip planner. Return days[] with stops (JSON)..."
OUTPUT	Validated plan, places matched on the map

⚠ **Watch out for:** invented places and impossible schedules. Check opening hours and travel time before showing any plan.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

Travel personality
Users pick a style (foodie, adventurer, slow traveler) that changes the planner's choices and voice.

"Surprise me" mode
A mystery day plan revealed one stop at a time.

Group compromise
Collect each friend's wishes and let the AI negotiate a plan everyone accepts.

Trip memories
After the trip, turn photos and notes into a travel diary.

Fridge to Recipe

Open the fridge. Get dinner.

02

DIFFICULTY ●●●●○ AI CORE Fridge photo to recipes

WHAT IS IT?

An app that turns what you already have at home into real meals. Take a photo of your fridge or type your ingredients, and it suggests recipes you can cook right now.

It tells you what is missing and adapts to your diet and time. Less food waste, fewer shopping trips.

MAIN FEATURES

- **Pantry** with quantities and expiry dates
- **Add items** by typing, receipt or fridge photo
- **Recipes ranked** by what you already have
- **Shopping list** of the missing ingredients
- **Preferences:** diet, allergies, equipment, time
- **Cooking mode** with steps and timers
- **History and ratings** of cooked meals
- **Expiry alerts** for food to use soon

HOW IT WORKS

1. **Show**
   Photograph or list what you have.
2. **Match**
   Get recipes ranked by what you can cook.
3. **Cook**
   Follow the steps; the pantry updates.

WHO USES IT

Home cook
Manages the pantry, finds and cooks recipes.

Household member
Shares the pantry and the shopping list.

Admin
Moderates shared recipes; watches AI cost.

SUGGESTED TECH STACK

Frontend	**React Native (Expo)** or **Next.js PWA**	Camera access for fridge photos and receipts.
Backend	**FastAPI** or **NestJS**	Endpoints for pantry, recipes and AI jobs.
Database	**PostgreSQL (+ pgvector, optional)**	Recipes and ingredients are many-to-many; similar-recipe search.
Services	**Vision LLM API, object storage, push**	Image understanding, photo storage, expiry reminders.
Testing & DevOps	**pytest/Jest, Docker, CI**	Test ingredient matching and unit conversion.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

- ✓ Accounts & login
- ✓ Image upload & storage
- ✓ Unit conversion (g, cup)
- ✓ Input validation
- ✓ Expiry reminders
- ✓ Rate limits on AI calls

Use Cases & AI Core

Fridge to Recipe

02

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Manage the pantry** Add, edit and remove ingredients with quantity and expiry.	Home cook
UC2	**Recognize ingredients from a photo** ✨ Upload a fridge photo; confirm or correct the detected items.	Home cook
UC3	**Get recipe suggestions** ✨ Ranked by available ingredients, time, diet and equipment.	Home cook
UC4	**Generate a custom recipe** ✨ Invent a new recipe from the current pantry on demand.	Home cook
UC5	**Adapt a recipe** ✨ Swap missing items, change servings or make it vegan.	Home cook
UC6	**Cook step by step** Cooking mode with timers; the pantry updates when you finish.	Home cook
UC7	**Build a shopping list** Missing ingredients from chosen recipes, grouped by aisle.	Home cook
UC8	**Share a household pantry** Members see and update the same pantry and list.	Household

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Vision:** a multimodal LLM reads a fridge photo and returns an ingredient list to confirm.
- **Recipe creator:** original recipes from your pantry as strict JSON (ingredients, steps, timers).
- **Smart substitutions:** "no butter?" gets ranked alternatives and their effect on taste.
- **Cooking assistant:** a chat that knows the current recipe ("too salty, now what?").
- **Waste saver:** weekly ideas that prioritize ingredients close to expiry.

EXAMPLE FLOW

INPUT	Fridge photo + "vegetarian, 30 min"
PROMPT	"List visible ingredients (JSON), then suggest 3 recipes..."
OUTPUT	Ingredients to confirm + 3 recipe cards

⚠ **Watch out for:** misread photos and unsafe advice (allergies, raw meat). Let users confirm every detected item.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

★ Chef personalities

Cook with a strict French chef, a calm grandma or a lazy student.

★ Leftover challenge

A weekly game: invent a dish from 5 random items.

★ Cuisine passport

Collect stamps for cuisines you cook; the AI suggests the next country.

★ Budget mode

Find the cheapest meal that still uses food about to expire.

Study Battle

Beat your friends. Learn something.

03

DIFFICULTY ●●●●●

AI CORE Questions & answer judging

WHAT IS IT?

A real-time quiz battle app. You challenge friends or random opponents on any topic you choose, such as the material of your next exam.

An LLM writes the questions from your notes, judges written answers and explains every mistake. Win battles to climb the leagues.

MAIN FEATURES

- **Battles** on a topic, uploaded notes or a syllabus
- **Real-time 1v1** and small-group matches
- **Question types:** choice, true/false, short text
- **Ratings, leagues** and seasonal rankings
- **Friends,** challenges and rematches
- **Review screen** that explains every mistake
- **Study stats:** weak topics, accuracy, streaks
- **Solo practice** and revenge on missed questions

HOW IT WORKS

1. **Choose**
   Pick a topic or paste your notes.
2. **Battle**
   Answer fast and correctly, live.
3. **Review**
   Learn from explanations; climb the league.

WHO USES IT

Player
Battles friends, earns rating, studies mistakes.

Host
Creates private arenas for a class or group.

Moderator
Reviews reported questions and abuse.

SUGGESTED TECH STACK

Frontend	**React (Vite) + Framer Motion** or **Flutter**	Fast, animated battle screens.
Backend	**Node.js + Socket.IO** or **NestJS WebSockets**	Real-time rounds and live scoring.
Database	**PostgreSQL + Redis**	Ratings in SQL; match state and leaderboards in Redis.
Services	**Auth.js, job queue, LLM JSON mode**	Generate questions before the match starts.
Testing & DevOps	**Vitest, Playwright, Docker, CI**	Simulate two players in one end-to-end test.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

✓ Accounts & login ✓ Real-time reconnect handling ✓ Server-side scoring (anti-cheat) ✓ Input validation ✓ Rate limits on AI calls ✓ Report & moderation flow

Use Cases & AI Core

Study Battle

03

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Sign up and build a profile** Account, avatar, rating and study statistics.	Player
UC2	**Create a battle topic** ✨ Choose a topic or paste notes to base the questions on.	Player, Host
UC3	**Find an opponent** Invite a friend or get matched by rating.	Player
UC4	**Play a live battle** Real-time rounds with timers and live score updates.	Player
UC5	**Judge written answers** ✨ The LLM grades short answers and explains its verdict.	Player
UC6	**Review a finished battle** ✨ Every question explained, linked to the source notes.	Player
UC7	**Climb the leaderboard** Ratings, leagues and seasonal rankings.	Player
UC8	**Report a bad question** Flag a question; a moderator reviews and removes it.	Player, Moderator

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Question generator:** notes or a topic become balanced questions with answer keys and difficulty.
- **Answer judge:** grades free-text answers, accepts paraphrases, gives partial credit.
- **Explanations:** a short reason and a concept hint after every question.
- **Adaptive difficulty:** next questions follow each player's mistakes so matches stay fair.
- **Quality guard:** a second pass checks each question has exactly one correct answer.

EXAMPLE FLOW

INPUT	Lecture notes on "TCP vs UDP"
PROMPT	"Create 10 questions (JSON) with answer, difficulty..."
OUTPUT	Validated question set, cached for the match

⚠ **Watch out for:** latency during live play and wrong answer keys. Generate and validate questions before the match starts.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

Boss battles
Fight an AI boss with its own personality and tricks.

Power-ups
Freeze the timer, 50/50, or buy an AI hint with points.

Team clash
Classes or friend groups compete as teams on a live scoreboard.

Story mode
A campaign where each chapter uses the player's own notes.

Explain It Differently

Same idea. Explained your way.

04

DIFFICULTY ●●●●○ | AI CORE Personalized explanations

WHAT IS IT?

An app for the moments when a concept just does not click. Paste a topic, a paragraph or a photo of a textbook page, and the app explains it again in another way: simple words, an analogy from your hobby, a story, or step by step.

You tell it what worked, and it learns how you understand best.

MAIN FEATURES

- **Input:** text, topic, document or photo of a page
- **Styles:** simple, analogy, story, step-by-step, expert
- **"Still confused?"** button for a new angle
- **Learner profile:** level, interests, favorite styles
- **Follow-up chat** that keeps the topic in context
- **Quick checks** to test your understanding
- **Library** of saved explanations with search
- **Feedback** on every answer to improve the next

HOW IT WORKS

1. **Paste**
   Give the app the confusing idea.
2. **Choose**
   Pick a style or let the app choose.
3. **Check**
   Answer a quick test and rate it.

WHO USES IT

- **Learner**
  Asks for explanations and tests understanding.
- **Teacher**
  Prepares class topics; sees which styles help.
- **Admin**
  Reviews flagged answers, quality and AI cost.

SUGGESTED TECH STACK

Frontend	**Next.js + React (streaming UI)**	Streaming text makes answers feel instant.
Backend	**FastAPI** or **NestJS**	Prompt building, streaming and file handling.
Database	**PostgreSQL (+ pgvector)**	Users, topics, explanations, feedback; similar-topic search.
Services	**LLM API (streaming), OCR/vision, storage**	Read photos and documents; keep the uploads.
Testing & DevOps	**pytest/Jest, prompt tests, Docker, CI**	Regression tests for prompts using fixed examples.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

✓ Accounts & login ✓ File & photo upload ✓ Streaming responses ✓ Input validation
✓ Library with search ✓ Usage caps per user

Use Cases & AI Core

Explain It Differently

04

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Create a learner profile** Level, interests and favorite styles, editable any time.	Learner
UC2	**Submit a concept** ✨ Paste text, enter a topic, or upload a document or photo.	Learner
UC3	**Get an explanation in a style** ✨ Simple, analogy, story, step-by-step or expert.	Learner
UC4	**Ask "Explain differently"** ✨ A new angle that avoids repeating what did not work.	Learner
UC5	**Chat with follow-ups** ✨ Ask more questions with the topic and history as context.	Learner
UC6	**Take a quick check** ✨ A short quiz from the explanation, with feedback.	Learner
UC7	**Manage the library** Save, tag, search and organize explanations.	Learner
UC8	**Review feedback and cost** Ratings per style, flagged answers and API usage.	Admin

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Multi-style explainer:** one prompt design, many styles, each with consistent structured output.
- **Personalization:** interests and past feedback shape prompts ("explain recursion with football").
- **Another angle:** the LLM sees earlier explanations and must take a genuinely different approach.
- **Understanding check:** generated questions estimate whether the learner really got it.
- **Grounding in files:** extract text or read a photo, then explain only what the source says.

EXAMPLE FLOW

INPUT	"Explain recursion" + profile: loves football, beginner
PROMPT	"Use a football analogy, 150 words, then 2 check questions"
OUTPUT	Streamed explanation + quick check

⚠ **Watch out for:** confident but wrong explanations. Ground answers in the source, show uncertainty, let users flag mistakes.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

Explain like I am...
Invent personas such as a pirate, a 5-year-old or a skeptical professor.

Concept map
Visualize how explained ideas connect and grow with each topic.

Voice mode
Talk to the explainer and listen to the answer on the go.

Teach-back
The learner explains the idea and the AI plays a confused student.

Gamified Learning Application

Study like it is a game.

05

DIFFICULTY ●●●●●○

AI CORE Quizzes & AI tutor

WHAT IS IT?

A learning app that turns studying into a game. Lessons become short challenges; you earn points, keep a daily streak, finish quests, unlock achievements and level up.

Pick a subject (for example programming basics) and make learning it feel fun.

MAIN FEATURES

- **Courses** split into short lessons and quizzes
- **XP, levels** and daily streaks
- **Daily quests** and achievement badges
- **Rewards shop** with avatars and themes
- **Friends,** leaderboards and weekly leagues
- **Progress dashboard** with weak topics
- **Streak reminders** by push notification
- **Content editor** for teachers

HOW IT WORKS

1. **Learn**
   Complete a short lesson or challenge.
2. **Earn**
   Collect XP and badges, keep the streak.
3. **Level up**
   Unlock levels and daily quests.

WHO USES IT

- **Learner**
  Takes lessons, earns rewards, competes with friends.
- **Teacher**
  Creates and edits lessons and questions.
- **Admin**
  Moderates content and watches analytics.

SUGGESTED TECH STACK

Frontend	**Next.js + Framer Motion** or **Flutter**	Smooth animations make rewards exciting.
Backend	**NestJS** or **Django**	A separate rules module for XP, levels and badges.
Database	**PostgreSQL + Redis**	Redis sorted sets for leaderboards and streaks.
Services	**Auth.js, push notifications, LLM API**	Accounts, streak reminders and the AI tutor.
Testing & DevOps	**Vitest/pytest, Docker, CI**	Unit-test the gamification rules thoroughly.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

✓ Accounts & login ✓ Progress saved in a database ✓ Rules engine with unit tests ✓ Input validation ✓ Push reminders ✓ Content moderation

Use Cases & AI Core

Gamified Learning Application

05

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Join and take a placement test** ✨ Pick a subject; a short test sets the starting level.	Learner
UC2	**Complete a lesson** Content plus quiz; XP awarded and progress saved.	Learner
UC3	**Earn rewards** XP, levels, streaks and badges update by clear rules.	Learner
UC4	**Receive daily quests** ✨ Personalized quests based on recent progress.	Learner
UC5	**Ask for an explanation** ✨ "Explain it simply" on any lesson or wrong answer.	Learner
UC6	**Compete with friends** Friend list, weekly leaderboard and challenges.	Learner
UC7	**Author a course** ✨ Write lessons by hand or draft them with AI, then review.	Teacher
UC8	**View learning analytics** Completion rates, hard questions and active users.	Admin

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Quiz generator:** lesson text becomes questions with explanations, as validated JSON.
- **AI tutor:** "explain it simply" in beginner or deeper mode, with hints that do not spoil.
- **Adaptive paths:** recent mistakes shape daily quests and review sessions.
- **Authoring assistant:** teachers draft lessons and questions with AI, then edit and approve.
- **Written-answer feedback:** short answers are graded with comments.

EXAMPLE FLOW

INPUT	Lesson text + learner level
PROMPT	"Write 5 multiple-choice questions (JSON)..."
OUTPUT	Validated quiz, cached for reuse

⚠ **Watch out for:** wrong answer keys and rising cost. Check that each question has one correct answer and cache quizzes.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

Study pet
A companion that grows and reacts to your habits, with an LLM-written personality.

Study guilds
Shared goals and weekly guild bosses.

Story campaign
Lessons become quests in an adventure written around the course topic.

Real-world quests
Challenges to apply knowledge offline, with photo proof.

Local Multiplayer Party Games

One phone. Many players. Zero boredom.

DIFFICULTY ●●●○○ AI CORE Fresh packs & game master

WHAT IS IT?

A mobile app with a collection of party games that friends play together on one single phone, passing it around.

Trivia, word challenges, guessing games and a social deduction game. No accounts and no setup: gather friends, pick a game, play.

MAIN FEATURES

- **Game hub** with 5+ mini games
- **Player setup:** names, avatars and teams
- **Rounds, timers,** scoring and results screen
- **Sounds, haptics** and animations
- **Offline packs** of questions and words
- **Custom packs** made and shared by players
- **Settings:** language, difficulty, family or adult mode
- **History** and hall of fame

HOW IT WORKS

1. **Gather**
   Add player names and pick a game.
2. **Pass the phone**
   Each player takes their turn.
3. **Score**
   See the scoreboard and crown the winner.

WHO USES IT

Host
Sets up the session and holds the phone.

Player
Takes turns, answers, bluffs and votes.

Pack creator
Builds and shares custom question packs.

SUGGESTED TECH STACK

Frontend	**Flutter** or **React Native (Expo)**	One app for Android and iOS, smooth animations.
Backend	**Small FastAPI** or **Firebase proxy**	Keeps API keys off the phone; games run without it.
Database	**SQLite** or **Hive / MMKV**	Local packs, scores and history for offline play.
Services	**Riverpod / Zustand, share codes**	Clean game state; share packs with a short code.
Testing & DevOps	**Widget/Jest tests, APK build**	Test the game rules; build an installable APK.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

✓ Works offline ✓ Local database ✓ Content rating settings ✓ Sound & haptics ✓ Rate limits on AI calls ✓ Accessibility (large text, contrast)

Use Cases & AI Core

Local Multiplayer Party Games

06

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Set up a game session** Add players and teams; pick game, rounds and difficulty.	Host
UC2	**Play a trivia round** Timed questions, scoring and streak bonuses.	Player
UC3	**Play a word or guessing round** Secret words, hints and a pass-the-phone flow.	Player
UC4	**Play a social deduction game** Hidden roles, discussion timer, voting and reveal.	Player
UC5	**Generate a question pack** ✨ Enter a theme and get a fresh pack saved for offline play.	Host
UC6	**Create and share a custom pack** ✨ Edit cards by hand or with AI; share with a code.	Pack creator
UC7	**Get game-master commentary** ✨ Funny commentary, dares and prompts between rounds.	Player
UC8	**View history and stats** Past sessions, scores and the hall of fame.	Host

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- ✨ **Pack generator:** any theme and difficulty becomes a validated JSON pack saved for offline play.
- ✨ **Game master:** commentary, nicknames and dares based on what just happened.
- ✨ **Deduction content:** secret words and prompts built from similar-but-different pairs.
- ✨ **Content safety:** family or adult rating enforced in prompts and by a moderation check.
- ✨ **Localization:** packs in the group's language and cultural context.

EXAMPLE FLOW

INPUT	Theme + difficulty + number of questions
PROMPT	"Create 10 trivia questions as JSON..."
OUTPUT	Pack saved on the phone for offline play

⚠ **Watch out for:** offline play and inappropriate content. Pre-generate and cache packs, and filter by the chosen rating.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

A brand-new game
Invent a game mode the LLM helps run, such as a storytelling relay.

Party personas
An AI host with a character: sports commentator, villain, robot.

Photo challenges
The AI describes or judges funny camera-based dares.

Seasonal events
Holiday packs that refresh themselves.

Mood-Based Music Discovery Application

Describe the feeling. Get the playlist.

07

DIFFICULTY ●●●●●○ | AI CORE Mood to music matching

WHAT IS IT?

Instead of searching for song names, you tell the app how you feel or what atmosphere you want (“rainy Sunday, a bit nostalgic”) and it shows songs that match.

Save favorites and group them into mood collections. Song data comes from an external music API.

MAIN FEATURES

- **Mood input:** free text, tags or emojis
- **Recommendations** with cover art and previews
- **“Why this song?”** short explanations
- **Mix controls:** energy, tempo, new vs. familiar
- **Mood collections** and favorites
- **Mood history** timeline
- **Share** a collection with a link
- **Daily mood mix** on the home feed

HOW IT WORKS

1. **Describe**
   Pick a mood or type how you feel.
2. **Discover**
   Browse tracks that fit the atmosphere.
3. **Save**
   Add songs to a mood collection.

WHO USES IT

- **Listener**
  Describes moods, previews and saves songs.
- **Curator**
  Builds and publishes public mood collections.
- **Admin**
  Moderates reports; watches API usage.

SUGGESTED TECH STACK

Frontend	**Next.js** or **Flutter**	Cover-art grids and an audio preview player.
Backend	**Express** or **FastAPI**	Gateway to the music API; hides keys, caches results.
Database	**PostgreSQL + Redis cache**	Users and collections in SQL; cache repeated API calls.
Services	**Spotify, Last.fm or Deezer API**	Search, tags, previews (check which endpoints new apps get).
Testing & DevOps	**Mock API tests, Docker, CI**	Test without hitting real rate limits.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

- ✓ Accounts & login
- ✓ External API with caching
- ✓ Rate-limit handling
- ✓ Input validation
- ✓ Privacy of mood history
- ✓ Share links

Use Cases & AI Core

Mood-Based Music Discovery Application

07

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Describe a mood** ✨ Free text, tags or emojis; the app understands the feeling.	Listener
UC2	**Get recommendations** ✨ Songs matched to the mood, each with a short reason.	Listener
UC3	**Refine the results** ✨ "More upbeat", "less vocals", "older songs".	Listener
UC4	**Preview and save tracks** 30-second previews and favorites.	Listener
UC5	**Manage mood collections** Create, edit, reorder and delete playlists by mood.	Listener
UC6	**Track mood history** A timeline of moods and what you listened to.	Listener
UC7	**Publish a public collection** Share with a link; others can follow it.	Curator
UC8	**Moderate content** Review reported collections and monitor API usage.	Admin

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Mood interpreter:** free text becomes a profile (genres, tags, energy, keywords).
- **Grounded retrieval:** songs come from catalog search; the LLM never names songs from memory unchecked.
- **Explain and re-rank:** a short reason per song; feedback re-ranks the list.
- **Conversational refinement:** tweak results over several turns, with memory.
- **Smart naming:** titles and summaries for mood collections.

EXAMPLE FLOW

INPUT	"Rainy Sunday, a bit nostalgic"
PROMPT	"Return genres, tags, energy 0–1 (JSON)..."
OUTPUT	Keywords sent to the music API

⚠ **Watch out for:** made-up titles and artists. Verify every track in the catalog before showing it.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

Mood of the day
Suggest a mood from weather, time and calendar (with consent).

Scene soundtrack
Describe a movie scene or photo and get its soundtrack.

Mood DJ
A radio that adapts as you say how you feel.

Mood cards
Friends send each other a mood card with a playlist.

Language Learning Application

A little practice every day.

DIFFICULTY ●●●●●○

AI CORE Tutor, coach & conversation

WHAT IS IT?

An app to build a new language step by step with flashcards, quizzes and daily challenges, at your own pace.

It remembers which words you find hard and brings them back at the right time, and an AI partner lets you practice real conversations.

MAIN FEATURES

- **Flashcards** with spaced repetition
- **Exercises:** choice, fill-in, listening, translation
- **Daily challenge,** streak and goals
- **Progress stats:** words, accuracy, time studied
- **Pronunciation** with text-to-speech
- **Custom decks** and import from texts
- **Conversation practice** with an AI partner
- **Reminders** and offline study mode

HOW IT WORKS

1. **Study**
   Flip through cards of new words.
2. **Practice**
   Answer exercises and daily challenges.
3. **Review**
   Hard words return before you forget them.

WHO USES IT

- **Learner**
  Studies, practices and tracks progress.
- **Tutor**
  Assigns decks and follows learner progress.
- **Admin**
  Manages content and watches AI cost.

SUGGESTED TECH STACK

Frontend	**React Native** or **Next.js PWA**	Mobile-first; good for short daily sessions.
Backend	**FastAPI** or **Django**	A clean place for the spaced-repetition algorithm (SM-2).
Database	**PostgreSQL**	Decks, cards and review history with next-review dates.
Services	**Browser TTS, Tatoeba, speech API**	Voice playback and real example sentences.
Testing & DevOps	**pytest, Docker, CI**	Test the scheduler with sample review histories.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

✓ Accounts & login ✓ Offline study mode ✓ Scheduler unit tests ✓ Input validation ✓ Daily reminders ✓ Rate limits on AI calls

Use Cases & AI Core

Language Learning Application

08

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Choose language and level** Onboarding with target language, level and daily goal.	Learner
UC2	**Study flashcards** Spaced repetition schedules each card's next review.	Learner
UC3	**Take a quiz** Mixed exercise types with instant feedback.	Learner
UC4	**Get examples and explanations** ✨ Example sentences and grammar notes for any word.	Learner
UC5	**Get writing feedback** ✨ Submit a short text; receive corrections and tips.	Learner
UC6	**Practice a conversation** ✨ Role-play (ordering coffee) with gentle corrections.	Learner
UC7	**Build a deck from a text** ✨ Paste an article; extract useful words into a deck.	Learner
UC8	**Assign decks and track progress** A tutor assigns decks and sees learner statistics.	Tutor

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Examples and explanations:** at the learner's CEFR level, in their native language.
- **Writing coach:** corrections, mistake categories and a rubric score as JSON.
- **Conversation partner:** role-play scenarios that adapt to the learner's level.
- **Personal exercises:** built from the words the learner keeps forgetting.
- **Deck builder:** vocabulary from any text, with translations and examples.

EXAMPLE FLOW

INPUT	Learner's short paragraph + level (A2)
PROMPT	"Correct it, explain each mistake (JSON)..."
OUTPUT	Corrected text, tips, rubric score

⚠ **Watch out for:** wrong grammar explanations and level drift. State the CEFR level in prompts and let users report errors.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

Persistent characters
Practice with AI characters who remember you and your last chat.

Real-world missions
"Order food in Italian today", then report back.

Learn from your life
Build lessons from the songs, shows or news you pick.

Pronunciation coach
Speech recognition with friendly feedback.

Digital Closet and Outfit Planner

09

Your wardrobe, one tap away.

DIFFICULTY ●●●●●○ | AI CORE Photo tagging & stylist

WHAT IS IT?

An app where you photograph and catalog your clothes, then combine them into outfits for different occasions.

Sort by category, color and season, save favorite looks, and never stand in front of the closet wondering what to wear.

MAIN FEATURES

- **Wardrobe catalog:** photo, category, color, season
- **Auto-tagging** from a photo
- **Outfit builder** canvas and saved outfits
- **Outfit calendar** that tracks what you wore
- **Daily suggestion** from weather and plans
- **Wear stats:** cost per wear, unused items
- **Packing lists** for trips
- **Wishlist** and wardrobe gaps

HOW IT WORKS

1. **Add**
   Photograph a garment and tag it.
2. **Combine**
   Build outfits from your own items.
3. **Wear**
   Pick a look for the occasion and weather.

WHO USES IT

- **Owner**
  Catalogs clothes and plans outfits.
- **Friend**
  Comments on shared outfits as a stylist.
- **Admin**
  Moderates reports; watches AI cost.

SUGGESTED TECH STACK

Frontend	**React Native (Expo)** or **Flutter**	Camera and gallery access for item photos.
Backend	**NestJS** or **FastAPI**	REST API for items, outfits and uploads.
Database	**PostgreSQL (tags as JSONB)**	Items, outfits and their links, with flexible tags.
Services	**Cloudinary/S3, Open-Meteo API**	Image storage and a free weather API.
Testing & DevOps	**Jest/pytest, Docker, CI**	Test outfit rules such as one top, one bottom.

WHAT MAKES IT FEEL LIKE A REAL PRODUCT

- ✓ Accounts & login
- ✓ Photo upload & storage
- ✓ Private user data
- ✓ Input validation
- ✓ Weather integration
- ✓ Rate limits on AI calls

Use Cases & AI Core

Digital Closet and Outfit Planner

09

USE CASES YOU MUST IMPLEMENT

Analyze, design and implement every one of them, then add your own. ✨ marks use cases that rely on the LLM.

UC1	**Add a clothing item** Photo, details and tags, stored privately.	Owner
UC2	**Auto-tag from a photo** ✨ A vision LLM proposes category, color and season; you confirm.	Owner
UC3	**Browse and filter the wardrobe** By category, color, season and tags.	Owner
UC4	**Build and save an outfit** Combine items on a canvas; rules check completeness.	Owner
UC5	**Get outfit suggestions** ✨ For the weather and occasion, using only items you own.	Owner
UC6	**Plan the week** Assign outfits to days and log what was worn.	Owner
UC7	**Pack for a trip** ✨ A packing list and outfits generated from the forecast.	Owner
UC8	**Ask for style feedback** ✨ Friends comment, or ask the AI for a style critique.	Owner, Friend

AI CORE • HOW THE LLM POWERS THE APP

CORE REQUIREMENT

- **Vision auto-tagging:** a multimodal LLM proposes category, color and season from a photo.
- **Grounded outfits:** suggestions return item IDs from your wardrobe, checked against the database.
- **Natural-language search:** "something blue for a wedding" finds matching pieces.
- **Gap analysis and critique:** what is missing, plus honest feedback on a look.
- **Trip packing:** a capsule list built from the forecast and planned events.

EXAMPLE FLOW

INPUT	Wardrobe (JSON) + 12°C, rainy, interview
PROMPT	"Suggest 3 outfits using only these IDs..."
OUTPUT	Outfits validated against the database

⚠ **Watch out for:** items that do not exist and private photos. Accept only known IDs, and keep user photos private.

MAKE IT YOURS

Creativity is a big part of your grade. Pick one direction, or invent your own.

Style DNA
Learn the user's taste from liked outfits and describe it.

Virtual stylist
Chat with stylist personalities, from minimalist to bold.

Sustainable closet
Encourage re-wearing, swaps and donation ideas.

Capsule wardrobe
Propose the few items that make the most outfits.

Ground Rules

What every project needs, whichever you choose.

BASELINE FOR EVERY PROJECT

VERSION CONTROL

Use Git with a shared repository, meaningful commit messages and short feature branches.

ANALYSIS AND DESIGN

Turn every use case into requirements, UML diagrams (use case, class, sequence) and a database schema.

EVERY USE CASE, IMPLEMENTED

All listed use cases must work end to end, and you add your own. Show each one in the demo.

AUTOMATED TESTS

Unit tests for core logic and integration tests for key flows, including AI features with mocked responses.

REAL-PRODUCT QUALITY

Accounts, saved data, input validation, and clear loading, empty and error states.

CODE REVIEW

Review each other's changes. Keep layers separate: interface, logic, data and AI.

CI AND EASY SETUP

A pipeline that builds and tests on every push; the app starts with one command.

DOCS AND DEMO

A clear README with setup steps, and a live demo of the finished product.

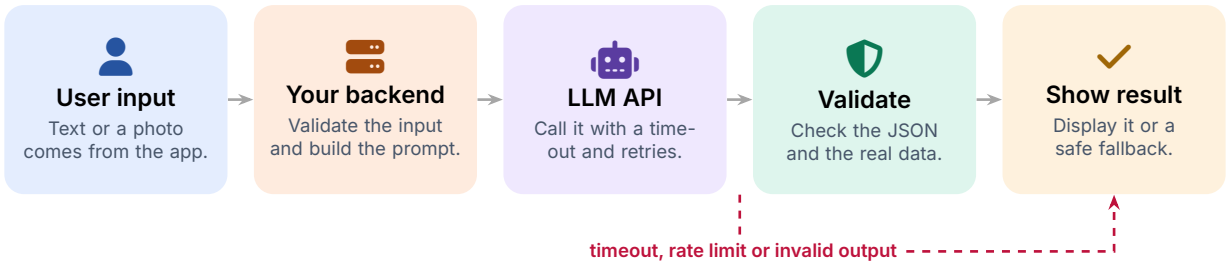
ANSWER THESE IN YOUR DESIGN DOCUMENT, FOR EVERY AI FEATURE

1. **The problem.** What user problem does the AI solve, and why is an LLM the right tool?
2. **The contract.** What goes in, and what exact structured output do you expect back?
3. **The checks.** How do you verify that the output is valid and based on real data?
4. **The failure plan.** What happens when the AI is slow, wrong or unavailable?
5. **The proof.** How will you measure that it is actually good: test prompts, examples, user feedback?

Working with LLM APIs

How to build the AI core responsibly.

A TYPICAL AI FEATURE PIPELINE



PROTECT YOUR API KEYS

Keep keys in environment variables, never in Git. Call the LLM from your backend, not from the client app.

CONTROL THE COST

Set a maximum number of tokens, cache repeated answers and handle rate-limit errors with retries.

EXPECT WRONG ANSWERS

LLMs can be confidently wrong. Ask for JSON, validate it, check it against your own data and label AI content.

TREAT USER TEXT AS DATA

Users may try “ignore your instructions”. Keep instructions separate; never run actions from LLM output unchecked.

SHARE ONLY WHAT IS NEEDED

Send minimal, anonymized data. Keep personal details out of prompts.

PLAN FOR FAILURE

APIs can be slow or down. Use timeouts and loading states, and keep the app usable without AI.

VERSION AND TEST YOUR PROMPTS

Keep prompts in files, not scattered in code, and test them with fixed example inputs.

LOG AND MEASURE

Log requests, latency, cost and failures so you can see what the AI feature really does.

Evaluation

What your project will be judged on.

Analysis and design — 20%

Clear requirements and use cases, sound UML models, a well-structured architecture and database.

AI integration — 25%

An AI core that is essential, well designed, validated and safe, with graceful failure handling.

Creativity and product thinking — 15%

Original ideas, thoughtful details and features that go beyond the list.

Implementation — 20%

Every use case working end to end in a polished, product-like application.

Engineering practice — 10%

Git history, automated tests, code review and a working pipeline.

Documentation and demo — 10%

A clear README and a confident final presentation.

Ready to choose?

Look back at the overview, pick the idea you would be proud to show, and think about the AI core and the creative twist that will make it yours.